

ICML 2023: One Dual Trick Makes Kernel PCA Faster, More Robust, and Sparser at Once

Researchers at KU Leuven recast Kernel PCA as a difference-of-convex problem and dualize it into a single, modular framework

Kernel PCA is one of the workhorses of unsupervised learning. By running principal component analysis in a kernel-induced feature space, it captures nonlinear structure that ordinary linear PCA cannot. But it carries a stubborn scaling problem: the standard recipe computes a singular value decomposition (SVD) of the n -by- n Gram matrix, and that $O(n^3)$ step turns painfully slow well before a dataset is anywhere near what most people would call large-scale data.

There is a second, quieter frustration. People often want a variant of Kernel PCA: one that is robust to outliers, or one that returns a sparse solution. Historically, each new property meant a new, purpose-built optimization problem and its own weighting heuristics. The result was a scattered collection of unrelated methods rather than a single coherent framework.

This paper from KU Leuven, presented at ICML 2023, offers a clean answer. Rewrite Kernel PCA as a difference-of-convex (DC) problem, then dualize it. That one move buys three things at once: a gradient-based solver that sidesteps the expensive SVD, a modular framework that swaps in robust or sparse objectives on demand, and substantial speedups on real benchmarks.

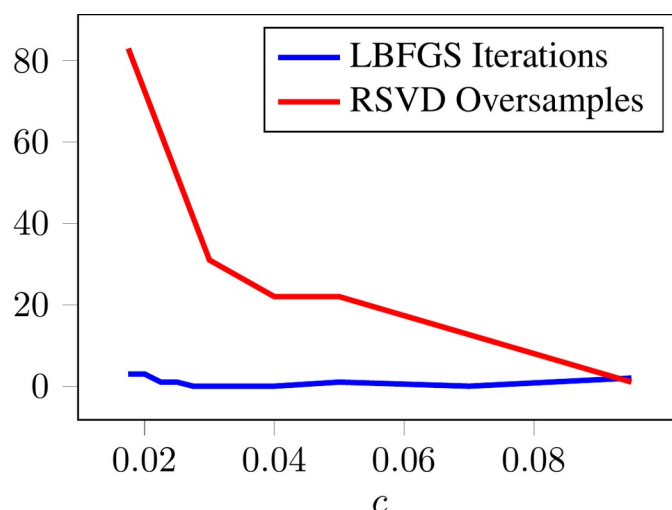


Figure 1. How eigenspectrum decay affects each solver: larger c means the spectrum of the Gram matrix G decays faster. As decay slows (smaller c), randomized SVD (red, RSVD Oversamples) needs sharply more oversamples to hit a fixed accuracy, while the paper's L-BFGS solver (blue, LBFGS Iterations) barely changes its iteration count. That flatness is why it stays ahead on real data.

Paper: <https://arxiv.org/abs/2306.05815>

Code: <https://github.com/taralloc/dc-kpca>

The problem: where classical Kernel PCA gets stuck

Solving classical Kernel PCA amounts to finding the leading eigen-directions of the n -by- n Gram matrix, an $O(n^3)$ SVD. Once n reaches the tens of thousands, that step is both slow and memory-hungry. The paper's timing study makes the cost concrete: on RCV1 ($n=20242$) and CIFAR-10 ($n=60000$), computing the full SVD to machine precision runs past a 30-minute cap and simply does not finish.

The second clue comes from those wished-for variants. Making Kernel PCA robust to outliers, or pushing it toward sparse solutions, has usually meant hand-designing a loss or a weighting scheme. Each construction stands alone, with no shared viewpoint to explain or solve them together. The paper sets out to fix both issues at the same time: fast, and able to absorb robustness and sparsity gracefully.

The core idea: write it as a difference of convex functions, then dualize

The starting point is an observation optimization researchers know well. Kernel PCA can be written as variance maximization under orthonormality constraints (a Stiefel manifold), which places it squarely in the family of difference-of-convex problems, where the objective is a difference of two convex functions, $\inf g(W)$ minus $f(\Gamma W)$. Earlier work had studied PCA through a DC lens, but only for linear PCA, and often only for the first component, where the orthogonality constraints vanish. This paper extends the treatment to the nonlinear, multi-component kernel case.

The pivotal step is dualization. In the primal problem the variable W lives in a possibly infinite-dimensional feature space, which rules out direct gradient descent. A general dualization proposition converts it into a dual problem whose variable H is a finite n -by- s matrix (s is the number of components to keep), which is exactly what gradient methods want. The indicator of the Stiefel manifold becomes, in the dual, a nuclear-norm term $\pi(H) = \text{Tr} \sqrt{H^T G H}$. Its gradient needs only the SVD of a small s -by- s matrix, never the full n -by- n one. $O(n^3)$ drops to $O(s^3)$ per iteration, and that is the source of the speedup.

For standard Kernel PCA, the dual is solved with L-BFGS. To induce robustness or sparsity, you add a Moreau-envelope term, written $\Psi\text{-star}(H)$, to the objective and optimize with a difference-of-convex algorithm (DCA), plugging in closed-form proximal operators for the Huber and epsilon-insensitive losses. Throughout, the method only ever touches small s -by- s SVDs, never the n -by- n one. In other words, robust and sparse are not three separate methods: they are the same dual framework with a different objective slotted in.

Payoff one: speed

The efficiency experiments span synthetic and real data: Synth 1 ($n=7000$), Protein ($n=14895$), RCV1 ($n=20242$), and CIFAR-10 ($n=60000$). All solvers share a single stopping criterion, a relative dual-cost residual, compared at two tolerances ($\delta = 1e-2$ and $1e-4$), with

timings averaged over 5 runs. The table below reports training time in seconds at the looser tolerance $\delta = 1e-2$, together with the speedup over randomized SVD.

Table 1. Kernel PCA training time (seconds) at $\delta = 1e-2$ for each solver, with speedup over RSVD. A value of >30 min marks a full SVD that exceeded the 30-minute cap. Numbers from the paper's Table 2.

Dataset	n	SVD	Lanczos	RSVD	Ours	Speedup
Synth 1	7000	96.73	0.85	1.97	0.53	3.72x
Protein	14895	868.64	3.46	6.70	1.07	6.25x
RCV1	20242	>30 min	6.04	12.50	2.12	5.90x
CIFAR-10	60000	>30 min	48.10	123.89	13.51	9.17x

The result is unambiguous: at $\delta = 1e-2$, the paper's solver beats full SVD, Lanczos, and randomized SVD on every task, and it is at least 3 times faster than RSVD, peaking at a 9.17x speedup on CIFAR-10, a scale at which the full SVD does not even finish inside the allotted 30-minute cap.

Figure 1 explains why that edge holds up on real data. Randomized SVD's accuracy depends on how quickly the Gram matrix's eigenspectrum decays. When the spectrum decays slowly, a common regime in practice, RSVD needs many more oversamples to reach the same accuracy, and its cost climbs. The paper's solver is largely indifferent to this: its iteration count barely moves.

What happens as s grows? Figure 2 studies the effect of the number of components s on random data with $n=15000$. Across a wide range, L-BFGS stays faster than RSVD. Only at very large s does the per-iteration s -by- s SVD begin to bite, and the two curves converge. For typical dimensionality reduction (s in the tens), that crossover sits well outside the useful range.

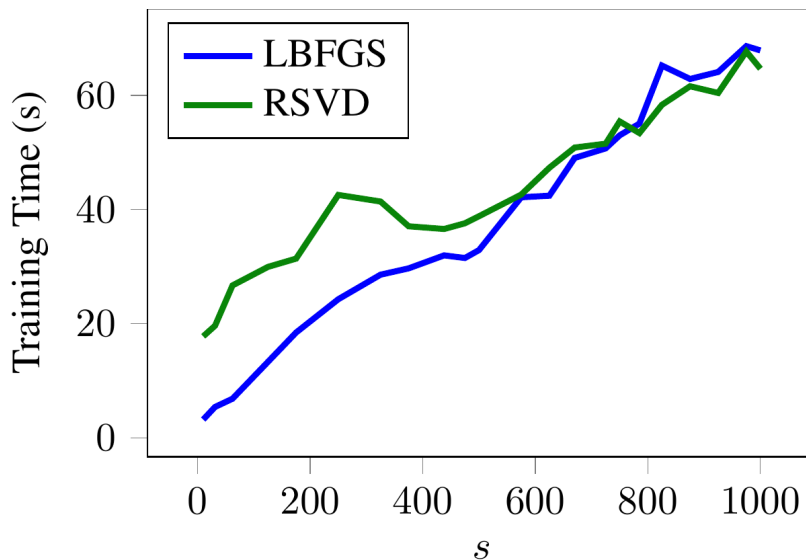


Figure 2. Effect of the number of components s on training time (random data, $n=15000$): both methods slow down as s grows, but L-BFGS (blue) stays ahead of RSVD (green) across a wide range. Only at very large s does the per-iteration s -by- s SVD pull the two together.

Payoff two: robustness

Robustness comes from replacing the squared loss in the objective with a Huber loss, which in the dual framework corresponds to adding a particular Moreau-envelope term. The paper considers two Huber variants, $H1_kappa$ and $H2_kappa$ (Moreau envelopes of an L1 and a per-sample L2 norm, respectively), both optimized with the same DC algorithm. The experiment runs on the Iris dataset contaminated with multiplicative Gaussian outliers, using a Gaussian kernel, and measures reconstruction MSE.

Figure 3 sweeps the Huber parameter $kappa$ and compares the two Huber curves against the red dashed line for the plain squared loss (half-norm-squared). Over a suitable range of $kappa$, both $H1_kappa$ and $H2_kappa$ push MSE below the squared-loss baseline, confirming that the framework really does induce robustness. The paper's Table 4 pins down the numbers: at contamination level $\tau = 10$, $H2_kappa$ reaches an MSE of 6.83, clearly below the squared loss's 7.59.

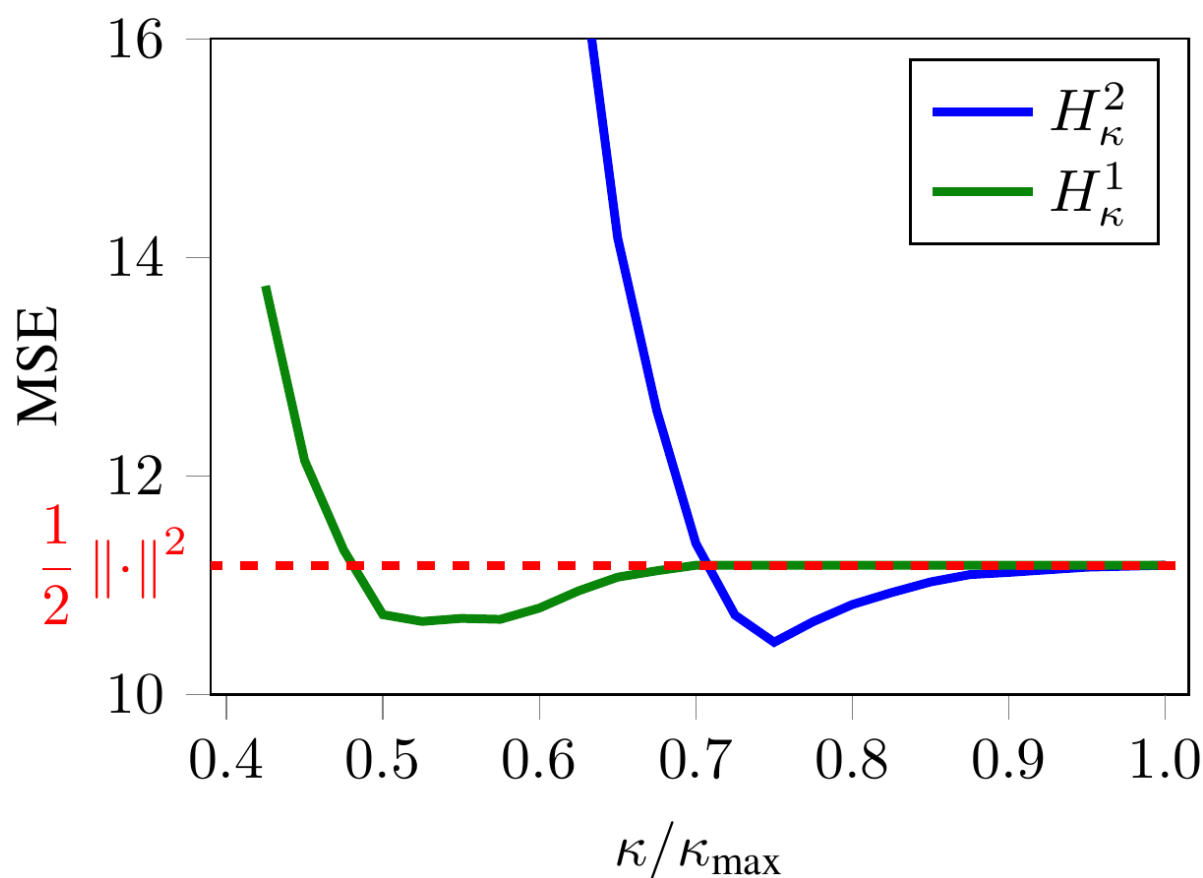


Figure 3. Robustness of the Huber losses (contaminated Iris, Gaussian kernel): the x-axis is the normalized Huber parameter $kappa/kappa_max$, the y-axis is MSE. The red dashed line is the squared-loss baseline; over a suitable $kappa$ range, both $H1_kappa$ (green) and $H2_kappa$ (blue) drop below it.

Payoff three: sparsity

Swap the same framework onto an epsilon-insensitive loss and you get sparse Kernel PCA: many dual coefficients are driven exactly to zero, so the principal components are expressed

using only a fraction of the samples. The natural cost is some loss of reconstruction accuracy, and the real question is whether that trade is worth it.

Figure 4 plots the reconstruction error ratio (relative to the dense solution) under the L2-epsilon loss as sparsity (the fraction of zeroed coefficients) increases, with color encoding the number of components s . The curves stay gentle: even at high sparsity, the error ratio rises only modestly. In other words, you can keep a fairly sparse solution in exchange for a mild drop in reconstruction quality, a controllable and interpretable accuracy-sparsity trade-off.

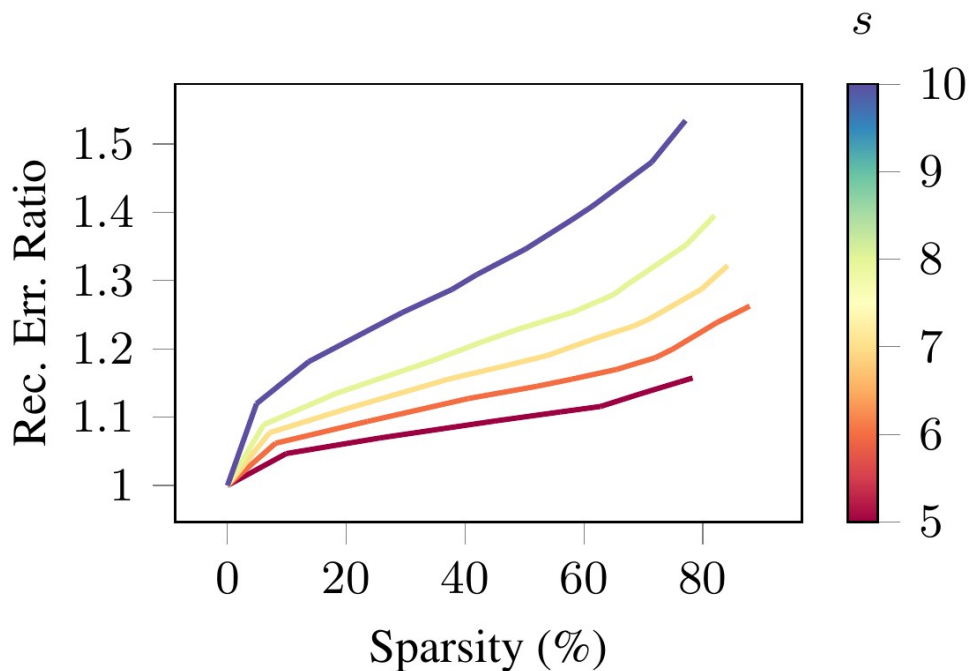


Figure 4. Accuracy-sparsity trade-off for sparse Kernel PCA (L2-epsilon loss): the x-axis is sparsity (fraction of zeroed coefficients), the y-axis is the reconstruction error ratio relative to the dense solution, and color encodes the number of components s . The gentle slope shows reconstruction quality degrades only modestly at high sparsity.

Takeaway: one viewpoint, three directions

The appeal of this paper is that a single change of viewpoint pays off in three directions. Once Kernel PCA is written as a difference of convex functions and moved to the dual, you get a solver that avoids the $O(n^3)$ SVD and runs up to roughly 9 times faster than randomized SVD. In the same breath, swapping the Moreau-envelope term in the objective lets you ask for robustness (Huber loss) or sparsity (epsilon-insensitive loss) without starting over.

One easily overlooked but important property: the dual variable H is always finite-dimensional (n -by- s), so the framework even covers infinite-dimensional feature maps that classical SVD-based Kernel PCA cannot handle directly. The boundaries are just as clear: when s grows very large, the per-iteration s -by- s SVD gradually eats into the speed advantage, and sparsity is bought with reconstruction accuracy, a trade to weigh per task. But as a way of folding fast, robust, and sparse into one dual optimization language, it offers a clean and extensible path.